

TECHNICAL REPORT

Course: Final Project - Data Visualisation (Track A & Track B)

Date: May 22, 2026

SECTION 1: EXECUTIVE SUMMARY

This report presents the design, technical implementation, and interpretive analysis of two distinct yet complementary data visualization systems: **Track A: Social Network Visualization & Community Discovery** and **Track B: Real-Time Stream Visualization Dashboard & Trading Simulator**. Together, these systems demonstrate how network science, modularity algorithms, real-time data streaming, and human-computer interaction (HCI) principles can be combined to reduce cognitive load and reveal hidden patterns in complex datasets.

Track A: Social Network Discovery (GoT Character Network)

- **Goal:** Perform a structural audit and find modular groups in a relational network of 39 key characters from *Game of Thrones* (bypassing the banned Zachary's Karate Club dataset).
- **Approach:** We applied the **Louvain Modularity** algorithm for community partitioning and scaled nodes using **Betweenness Centrality**. The results were rendered in a custom dark-mode force-directed network using PyVis.
- **Key Findings:** The network split into 5 distinct communities matching geopolitical factions. **Jon Snow** and **Tyrion Lannister** emerged as the primary "bridges" (highest betweenness). Simulating their removal fragmented the network, isolating the Essos campaign from Westeros.

Track B: Real-Time Stream Ingestion Dashboard

- **Goal:** Build a live financial monitoring dashboard that tracks Bitcoin (BTC), Ethereum (ETH), Solana (SOL), and Cardano (ADA) prices in real-time, focusing on velocity, temporal context, and alerting.
- **Approach:** Developed a Python-based **Streamlit** dashboard pulling live data from the **CoinGecko API** every 10 seconds. We plotted a 15-minute sliding window line chart using **Plotly**, implemented threshold alerting, and built an interactive Paper Trading Simulator with a \$100M balance and a real-time Profit/Loss (Win/Loss) scorecard.
- **Key Findings:** By locking Plotly's Y-axis to the sliding window's range and using muted base colors, we maintained visual stability. A user-configurable alerting system uses flashing CSS indicators to capture immediate attention. The P&L scorecard dynamically guides the user by rendering a green "WINNING" state or a red "LOSING" state based on total realized and unrealized returns.

SECTION 2: VISUALIZATION DESIGN DOCUMENT (VDD)

2.1 User Personas

To guide our visual and interactive choices, we designed both systems for specific user personas:

Persona 1 (Track A): Gillian Vance - Screenplay Auditor & Narrative Designer

- **Bio:** Gillian audits complex scripts for major television networks. She needs to ensure character prominence and factions align with the writer's narrative goals.
- **Goal:** Visually audit character factions and verify that main characters are structurally central, while minor characters remain on the periphery.
- **Pain Point:** Static "hairball" network charts cause cognitive overload and are unreadable.
- **How Track A Helps:** The custom interactive HTML layout allows Gillian to zoom, search characters, and click on nodes to fade out unrelated connections, revealing clean ego-networks.

Persona 2 (Track B): Marcus Vance - Cryptocurrency Day Trader

- **Bio:** Marcus is a day trader who monitors price fluctuations of volatile crypto assets to make rapid, split-second buy/sell decisions.
- **Goal:** Monitor price trends, check connection latency, and set alerts to be notified immediately when a price crosses a specific threshold.
- **Pain Point:** Dashboards with erratic, jumping axes disorient him, and standard alerts are easy to miss on busy screens.
- **How Track B Helps:** Plotly's locked y-axis margins prevent chart jumping, and the high-contrast flashing red alert banner ensures he never misses a threshold crossing.

2.2 Visual Encoding Mappings

Visual encoding maps dataset variables (data attributes) to pre-attentive visual variables of the graphical marks.

Track A	Character (Node)	Entity	Circular Dot	Neutral geometric mark suitable for scaling and coloring.
---------	------------------	--------	--------------	---

Track A	Character Prominence	Quantitative	Node Size (Radius)	Establishes immediate visual hierarchy; larger nodes indicate key actors.
Track A	Discovered Alliance	Categorical	Color Hue (Dracula)	Curated, high-contrast colors allow instant community classification.
Track A	Interaction (Edge)	Relational	Curved Line (Bezier)	Curves prevent straight lines from overlapping in dense cliques.
Track A	Interaction Strength	Quantitative	Edge Thickness	Thicker lines represent stronger relationships (frequent dialogue).
Track A	Sub-graph Focus	Relational	Opacity / Saturation	Muting non-connected nodes (15% opacity) eliminates visual noise on click.
Track B	Current Asset Price	Quantitative	Large Numeric KPI	Large fonts catch attention; green/red text indicates 24h gain/loss.
Track B	Price Trend (15m)	Temporal	Line Chart (X-Time, Y-Price)	Displays historical direction, rate of change, and volatility.
Track B	Price Alert	Boolean (Threshold)	Flashing Red Background	Pulse-animating red is a strong pre-attentive cue signaling urgent change.
Track B	Ingestion Status	Categorical	Status Badge (Green/Yellow)	Indicates system health (Live, Paused, or Rate-Limited/Cached).
Track B	Ingestion Latency	Quantitative	Metric Text (ms)	Measures API response time for connection audits.
Track B	Trade P&L Status	Categorical	Color Hue & Text	Green (WINNING) or Red (LOSING) highlights overall simulation success.

SECTION 3: PART 1 - SOCIAL NETWORK AUDIT (TRACK A)

3.1 Layout Justification

We utilized a **Force-Directed Layout** (specifically the **ForceAtlas2** physics solver). This layout behaves like a system of springs, where nodes repel each other and edges pull connected nodes close. This is optimal because it naturally groups dense communities together while spreading out weakly connected factions, making modularity visually apparent.

3.2 Community Narrative (Modularity Analysis)

The Louvain algorithm detected **5 modular communities**:

1. **The Northern Coalition (Stark Core) - Ice Blue**: Jon Snow, Ned, Robb, Sansa, Arya, Bran, Catelyn. Densely connected, representing the family unit in Winterfell.
2. **The Court at King's Landing (Lannister Core) - Gold/Orange**: Tyrion, Cersei, Jaime, Tywin, Joffrey, Bronn, Shae. Densely interconnected, representing the political plotting in the capital.
3. **The Essos Campaign (Targaryen Exile) - Red**: Daenerys, Jorah, Khal Drogo, Missandei, Grey Worm, Daario, Barristan. Geographically isolated, grouped tightly around their queen.
4. **Night's Watch & Wildlings - Green**: Samwell Tarly, Jeor Mormont, Maester Aemon, Ygritte, Tormund, Mance Rayder, Gilly. Bridges the military order and the free folk.
5. **House Baratheon & Rebels - Purple**: Stannis, Davos, Melisandre, Renly, Robert Baratheon, Brienne of Tarth. Captures the claimants to the throne.

3.3 "Weak Link" & Bridge Analysis

Using **Betweenness Centrality**, we identified the critical bridges:

- **Jon Snow (Centrality: 0.3883)**: Sits between the Starks, the Wall, and the Targaryens.
- **Tyrion Lannister (Centrality: 0.2646)**: Connects the Lannisters, the Starks, and the Targaryens.

Structural Simulation (Removing the Bridges):

If we remove **Jon Snow** and **Tyrion Lannister** (and their bridging edges), the graph splits into **two disconnected components**. The entire Essos/Targaryen community becomes an isolated island with **no path** to Westeros, and the Wall is cut off from Winterfell. This mirrors the narrative: without these negotiators, the factions would remain isolated, leading to structural collapse against external threats.

3.4 Visualization Critique & Noise Reduction

- **The Critique (Noise)**: The Lannister court forms a highly connected clique where overlapping lines create a dense web.
 - **Applied Solutions**:
6. **Label Proportionality**: We hide text labels on canvas for minor nodes, displaying them only on hover.

7. **Connected Opacity:** Clicking any node fades out all non-connected nodes.
8. **Bezier Curves:** Curved lines separate overlapping paths.

SECTION 4: PART 2 - REAL-TIME VISUALIZATION (TRACK B)

4.1 Data Pipeline & Ingestion Architecture

- **Source:** CoinGecko API (`simple/price` endpoint).
- **Ingestion Method:** REST Polling (Pull logic) using the python `requests` library.
- **Polling Frequency:** Every **10 seconds**. This is justified because it provides near-real-time price updates while staying safely below CoinGecko's rate limit of 30 requests/minute, avoiding HTTP 429 errors.
- **Latency Auditing:** Every API call is timed using python's `time.time()`. The response latency is displayed on the dashboard in milliseconds (`Latency: Xms`), serving as connection metadata.
- **Graceful Exception Fallback:** If the server hits a rate limit (HTTP 429), a custom caching layer intercepts the exception. It retrieves the last successfully fetched prices from `st.session_state` and renders a yellow warning banner explaining the rate limit, preventing app crashes.

4.2 The Temporal Window (Sliding Window) & Axis Stability

- **Sliding Window:** Set to a **15-minute sliding window**. For a 10-second polling frequency, the database retains exactly the last **90 data points** ($15 \text{ min} * 60\text{s} / 10\text{s}$). Older data points are popped from the array head as new ones are appended.
- **Axis Locking & Padding:** Plotly charts are prone to "jitter" (constant jumping of grid lines as the Y-axis min/max updates). To prevent this, we lock the Y-axis range dynamically on every frame to `[min_price - padding, max_price + padding]`, where padding is exactly 5% of the range.
- **Dual-Axis Asset Separation:** Bitcoin's price (\$95k) is orders of magnitude higher than Ethereum (\$3k), Solana (\$180), and Cardano (\$0.50). To prevent compression (which renders minor coins as flat lines), we plot Bitcoin on **Y-Axis 1 (Left)** and the other selected coins on **Y-Axis 2 (Right)**. A multi-select widget allows the user to toggle traces on/off.

4.3 Alerting & Threshold Logic

- **Configuration:** The user sets alert threshold values (e.g. SOL > \$200) inside the sidebar settings expander.
 - **Visual Trigger:** When a price tick exceeds the threshold:
9. The target KPI card's class dynamically switches to `.kpi-card-alert`, turning its border red and starting a CSS keyframe pulse animation.
 10. The price font changes to red.
 11. A flashing red warning banner is injected at the top of the viewport.

- **UX Rationale:** Marcus does not need to read individual digits; the color change and motion (pulsing) act as immediate pre-attentive alerts, notifying him of a threshold event.

4.4 Simulation Dashboard Design & UX

- **Simpler Layout:** The main interface is divided into two tabs:
 12. **Live Portfolio & Trading:** Focuses on prices, win/loss status, and asset holding tables.
 13. **Charts & Logs:** Houses the Line Chart, transaction logs, and CSV download.
- **Win/Loss Scorecard:** Calculates total profit/loss as $(\text{Total Assets Value} + \text{Cash}) - \text{Initial Cash}$. If the return is positive, it glows green and displays 🟢 WINNING! 🎉; if negative, it glows red and displays 🟡 LOSING! 📉.
- **Button Feedback:** When the user clicks "Execute Trade" or "Add Funds," a floating toast pop-up (`st.toast`) triggers immediately at the bottom right. A persistent "Last Action Feedback" banner is displayed at the top of the main container, confirming the transaction and timestamp.
- **Export CSV:** Users can export the recorded 15-minute session price history as a CSV file to audit price fluctuations offline.

4.5 Performance & Future Scaling

- **Current Performance:** Handles 1 call/10 seconds easily. Latency is negligible (<100ms API response time).
- **What Breaks at 1,000 updates/second?**
 14. **API Rate Limiting:** The CoinGecko free REST API would instantly block us with HTTP 429 errors.
 15. **Browser Rendering (CPU):** Streamlit re-renders the page on every tick. At 1,000 updates/sec, it would crash the browser due to CPU exhaustion from Plotly re-drawing.
 16. **Memory Leak:** Appending 1,000 points/sec to memory would quickly exhaust browser RAM.
- **How to Fix (Scaling Architecture):**
 17. **Ingestion:** Swap REST Polling for a **WebSocket Connection** (push-based streaming) to receive a continuous price feed.
 18. **Data Pipeline:** Add a message queue (**RabbitMQ** or **Apache Kafka**) and store data in a time-series database (**InfluxDB** or **TimescaleDB**).
 19. **Visualization (Rendering):** Switch from Streamlit/Plotly to a client-side **WebGL-rendered charting library** (like uPlot or custom D3 with Canvas/WebGL) to handle high-frequency plotting at 60 FPS without UI freezing.
 20. **Frontend Architecture:** Move from Streamlit to a decoupled Single Page Application (React/Svelte) with a backend API (Go/NodeJS) to isolate page rendering from background ingestion threads.